

LAB # 03

Designing Your Web Pages Using CSS-I

OBEJCTIVE:

- Introduction to CSS
- CSS Features
- CSS Core Syntax
- Style Sheets & HTML
- CSS Box Model
- Normal Flow Box Layout
- Beyond the Normal Flow

CSS is an acronym which stands for Cascading Style Sheets. **Cascading Style Sheets** is a style sheet language used for describing the presentation of a document written in a markup language.

A **style** may be applied to an existing HTML tag or can be used to create the equivalent of one.

Why use CSS?

- an alternative to programs which use templates
- give a consistent look and feel to page elements
- more control over layout
- should decrease page load time
- one file will change the look of your entire website

Applying CSS

There are three ways in which to apply css code to an HTML doc:

1. Inline
2. Internal
3. External

Inline

The style is used with a single instance of an HTML tag. In reality, it would make much more sense to use HTML to change only one specific tag

```
<html>
<head>
  <title>Inline Style Page Example</title>
</head>

<body style="background-color:#99FFCC;">

<p>This is an example where the background colour of this particular webpage would
be light blue</p>

</body>

</html>
```

Internal

Styles are defined in the head section of an HTML document. These styles only affect the instances included in the particular page. It is more common for people to use External style sheets.

```
<html>
<head>
  <title>Internal Style Page Example</title>
  <style type="text/css">
    body{background-color:#99FFCC;}
  </style>
</head>

<body>

<p>This is an example where the background colour of this particular webpage would
be light blue</p>

</body>

</html>
```

External

Styles are defined in a separate document called a "style sheet". This sheet may be referenced from many HTML documents. As such, each time the styles are changed within the sheet the "look" is reflected in HTML pages referencing the sheet.

```
<html>
<head>
  <title>External Style Page Example</title>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>

<body>

<p>This is an example where the background colour of this particular webpage would
be light blue</p>

</body>

</html>
```

Cascading

Different style sheets will overwrite (cascade) similar styles:

- ☐ inline overwrites internal
- ☐ internal overwrites external
- ☐ external overwrites browser default

Coding Syntax



Examples:

```
p {text-align:center; color=green;}
```

- more than one declaration may be referred to in a selector
- each declaration must be separated by a semi-colon (;) h1, h2, h3 {color:red;}
- you can apply the same property and value to various selectors as in the example above

Selectors

In CSS, a selector is the part of a rule that comes before the first "{", such as "p" in the rule "p {font-weight:bold; }".

A selector specifies to which elements a rule should apply, by naming the type of the element, such as "div", the class of the element, or the id of the element. Selectors can only be used in linked and embedded CSS, not in inlined one.

The following table provides an overview of selectors. The sections that follow discuss each type of selector in detail.

Type of Selector	Example Selector	Example Rule
type	div	div { margin:7px; padding:7px; } </pre>
class	.important	.important { font-weight:bold; color:red; }
id	#onlyThisOne	#onlyThisOne { font-family:courier; }
universal	*	* { color:green; }
pseudo-class	a:link	a:link { color:blue; }
pseudo-element	p:first-letter	p:first-letter { color:red }
descendant	td span	td span { font-weight:bold; }
grouped	h1, h2, h3	h1, h2, h3 { text-align:center; }

Type

These selectors match elements based on the name of their element type. The example above is using a type selector to make every instance of p have bold text. You can use the type selector with any element. Here are a few examples:

```
div {  
    margin:7px;  
    padding:7px;  
}  
body{  
    background-image:url("image.gif");  
    font-size:.9em;  
}
```

Tag Selectors

Tag selectors are those which change already in existence HTML tags.

```
p { background-color: red; }
```

Class Selectors

Class selectors are essentially a way in which to style an element which you name. For example, you might wish to create a type of text box for news stories on a webpage to be reused many times, or a way in which to wrap text around an image. The point is that a class selector can be used many times.

```
#leftcontainer  
{  
    background:url(images/template/shadow_left.gif) repeat-y right top;  
    width:419px;  
    float:left;  
    color:#585858;  
    font:11px Tahoma, serif;  
    height: 466px;  
    overflow: auto;  
}
```

ID Selectors

An ID selector is exactly like a class selector except that it can only be used ONCE in a page.

```
CSS rule:  
#onlyThisOne {  
    font-family:courier;  
}  
Sample HTML:  
<p id="onlyThisOne">Courier</p>
```

Universal

These selectors apply a style to all the items on the page. For example to make all the text on the page green use the

```
rule:
* {
color:green;
}
```

Pseudo-Classes

Pseudo-classes and pseudo-elements allow elements to be formatted on the basis of information other than the position of the element in the document tree. Pseudo-class and pseudo-element selectors are prefixed by a colon, :, in CSS1 and CSS2.1. In CSS3 pseudo-elements are prefixed by ::. CSS level 1 defines three pseudo-classes:

- link**
- unvisited links**
- visited**
- visited links**
- active**
- active links**

These can only be applied to the anchor (a) elements.

```
a:link{
color:blue;
}
```

Cascading Style Sheets/Selectors 5

```
a:visited{
color:purple;
}
a:active{
color:red;
}
```

CSS level 2.1 introduces several additional pseudo-classes, including hover. hover can be used to create rollover effects without resorting to Javascript. CSS2.1 also allows active to apply to any element that can be active, e.g. a button.

Descendant

Descendant selectors allow you to apply style to elements that are nested within another specified element. For example, you could have every span element inside of every p element be bold. The span can be a child element of the p element, or a grandchild, or a great-grandchild, etc.

CSS rule:

```
p span{
  font-weight:bold;
}
```

Sample HTML:

```
<p>Start of paragraph. <span>This span is bold.</span> Rest of paragraph.</p>
<div>Start of division. <span>This span is normal.</span> Rest of division.</div>
```

Example rendering:

Start of paragraph. **This span is bold.** Rest of paragraph.

Start of division. This span is normal. Rest of division.

The next example changes the color of visited links in the navigation section. Visited links in other parts of the document would be unaffected.

CSS rule:

```
ul#navigation a:visited {
  color:red
}
```

Sample HTML:

```
<ul id="navigation">
  <li><a href="HomePage">Home</a></li>
  <li><a href="AnotherPage">A page you haven't visited yet.</a></li>
</ul>
```

Grouping selectors

Selectors can be grouped into comma separated lists.

```
h1, h2, h3 { text-align:center; }
is equivalent to
h1 { text-align:center; }
h2 { text-align:center; }
h3 { text-align:center; }
```

An element may be selected by more than one rule:

```
h1, h2, h3, h4, h5, h6 {
  color: white; background-color: red; }
h1 {
  color:yellow
}
```

All headings will have a red background. Level 2–6 headings will have white text. Level 1 headings will have yellow text

HTML Block and Inline Elements

Div vs Span

Normally, creating the layout of a page should be done first; however, because you are learning different aspects of .css which are related, this is where we will accomplish the layout of the html page. So...as we are working on creating a page which will look something like the below

page, the following "boxes" have been drawn to show you what we are working on. Each box we are going to create is named and outlined as well as one big container for the entire page is indicated by arrows.

Div

- A block-level element always starts on a new line and takes up the full width available (stretches out to the left and right as far as it can).
- The `<div>` element is a block-level element.



Span

- An inline element does not start on a new line and only takes up as much width as necessary.
- The `<span>` element is an inline element.

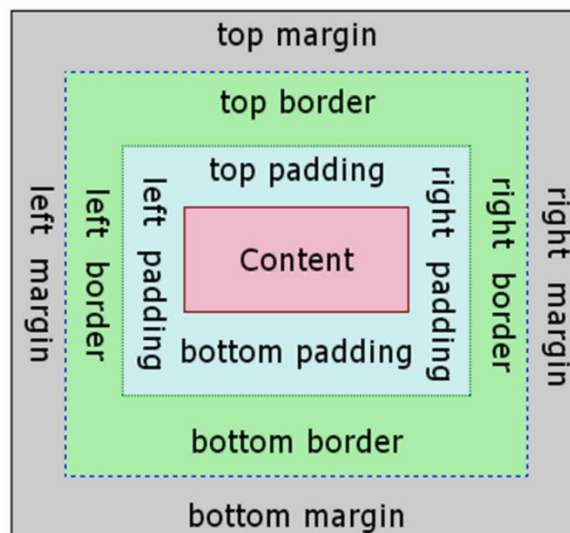


The CSS box model

The CSS box model is the foundation of layout on the Web — each element is represented as a rectangular box, with the box's content, padding, border, and margin built up around one another like the layers of an onion. As a browser renders a web page layout, it works out what styles are applied to the content of each box, how big the surrounding onion layers are, and where the boxes sit in relation to one another. Before understanding how to create CSS layouts, you need to understand the box model

Content of the box, where text and images appear

Border that goes around the padding and content. By default the border has a size of 0



Padding refers to the *inner* margin of a CSS box — between the outer edge of the content box and the inner edge of the border.

Margin extends the border area to include an empty area used to separate the element from its neighbors

width and height

The **width** and **height** properties set the width and height of the **content box**, which is the area in which the content of the box is displayed — this content includes both text content set inside the box, and other boxes representing nested child elements.

padding

Padding refers to the *inner* margin of a CSS box — between the outer edge of the content box and the inner edge of the border. The size of this layer can be set on all four sides at once with the **padding** shorthand property, or one side at a time with the **padding-top**, **padding-right**, **padding-bottom** and **padding-left** properties.

border

The **border** of a CSS box sits between the outer edge of the padding and the inner edge of the margin. By default the border has a size of 0 — making it invisible — but you can set the thickness, style and color of the border to make it appear. The **border** shorthand property allows you to set all of these on all four sides at once, for example `border: 1px solid black`. This can be broken down into numerous different longhand properties for more specific styling needs:

- **border-top**, **border-right**, **border-bottom**, **border-left**: Set the thickness, style and color of one side of the border.
- **border-width**, **border-style**, **border-color**: Set only the thickness, style, or color individually, but for all four sides of the border.
- You can also set one of the three properties of a single side of the border individually, using **border-top-width**, **border-top-style**, **border-top-color**, etc.

margin

The margin surrounds a CSS box, and pushes up against other CSS boxes in the layout. It behaves rather like padding; the shorthand property is **margin** and the individual properties are **margin-top**, **margin-right**, **margin-bottom**, and **margin-left**. This working of this box model can be understood by following example;

First, the HTML:

```
<div class="one"> This is first div</div>
<div class="two">This is second div</div>
```

Now the CSS:

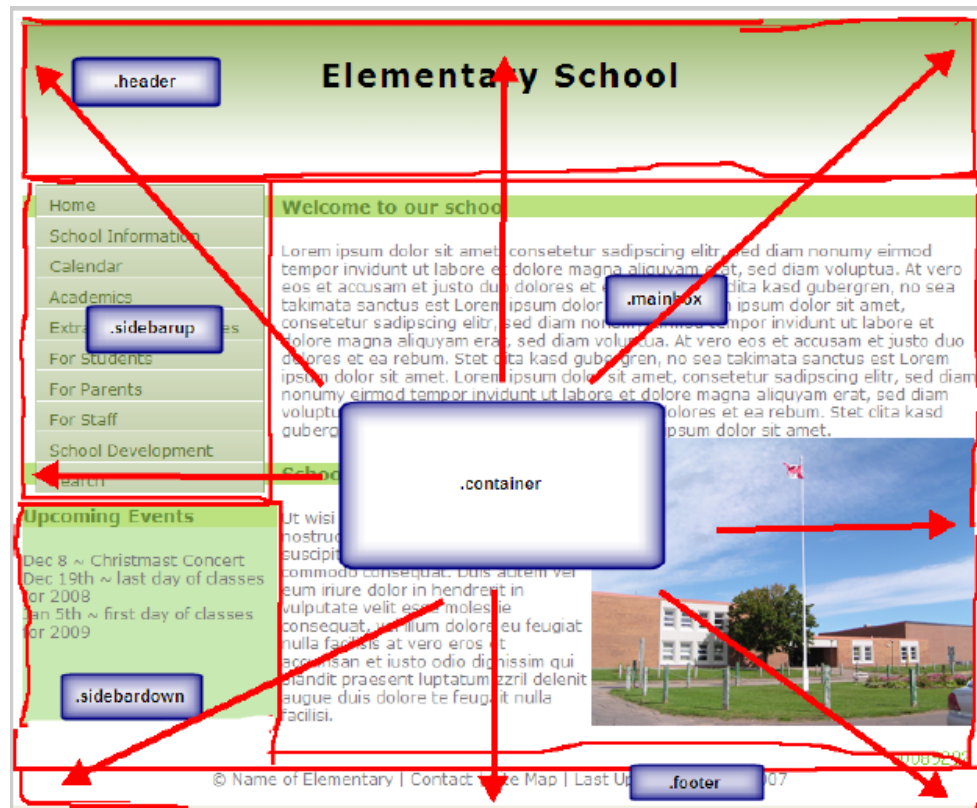
```
html {
  font-family: sans-serif;
  background: #ccc;
}

.one, .two {
  background: red;
  width: 300px;
  height: 150px;
  padding: 20px;
  border: 10px solid black;
  margin: 20px auto;
}

.two {
  box-sizing: border-box;
}
```


For more details please follow the following link;

[https://developer.mozilla.org/en-US/docs/Learn/CSS/Introduction to CSS/Box model](https://developer.mozilla.org/en-US/docs/Learn/CSS/Introduction%20to%20CSS/Box_model)



So...with keeping organization in mind in your styles.css page create a new header for the Class Selectors you are going to create to accomplish the layout. Creating a container to hold everything will accomplish what you did in the HTML section with the main table.

```

/**Layout**/

.container {
    float:none;
    width:750px;
}

.header {
    font-size: 24px;
    font-weight: bold;
    color: #000000;
    letter-spacing: 2px;
    text-align: center;
    height: 120px;
    width: 750px;
    padding-top: 30px;
    padding-right: 0px;
    padding-bottom: 30px;
    padding-left: 0px;
}

```

The .header style (which we will do more with in the future) basically replaces the top cell of the table.

The following 2 classes -- sidebarup and sidebardown create the equivalent of two cells which we will ensure make up the left hand side of the page.

```
.sidebarup {  
    text-align: left;  
    clear: left;  
    float: left;  
    width: 200px;  
}  
  
.sidebardown {  
    width: 200px;  
    float: left;  
    text-align: left;  
    clear: left;  
}
```

The .mainbox class creates a container which can mimic the main cell of the table created in HTML. There are some interesting features here which we will further exploit in a few lessons. The footer replaces the bottom cell of the table. Again...we will be adding to this soon.

```
.mainbox {  
    height: 500px;  
    width: 550px;  
    float: right;  
    text-align: left;  
}  
  
.footer {  
    width: 750px;  
}
```

Obviously, these styles will not yet affect any content on your webpage as they have not been coded in to the HTML document. So...because you are inserting totally NEW codes this is an appropriate time to talk about the "div" and "span" code. In order to insert these codes you will either use:

<div class="nameofstyle"></div>

OR

So when do you use div and when do you use span?

In short, div is used for block elements and span is used for inline elements. Block elements are items which stand on their own on a page and take up the horizontal space on either side of it, where inline elements are included in the line or flow of the page. Think of a block element as being similar to the paragraph in HTML (except it can contain more than just text) and the inline element would be similar to a font or link element where simply some text is changed.

LAB TASKS

TASK # 1

Create a web page using HTML as shown in figure 1:

Jake's Cofee Shop

[Home](#) [Menu](#) [Music](#) [Jobs](#)

Come in and experience...



- Specialty Coffee and Tea
- Freshly made sandwiches
- Bagels, Muffins, and Organic Snacks
- Music and Poetry Readings
- Open mic nights
- ...

23 Pine Road
Nottingham, NG1 5YU
0115 9324567

Copyright © 2011 Jake's Coffee House
jake@jcoffee.com

Figure 1: Display

Write the appropriate **CSS** to change the presentation of the webpage to that shown in **figure 2**. The new page layout includes a header section, two columns (one for navigation menu and one for the main content) and a footer section.



The **colors** used are as follows:

Text: #330000

Body background: #ffffcc

Header and footer background: #ccaa66

Left (navigation) column background: #E8D882

Right (main content) column background: #f1e8b0

The **text** is displayed in Verdana font.

You should write an external CSS file. You need to modify the original HTML file only to link to this style sheet, mark up sections (using <div> tags) and set appropriate class and id attributes.

TASK # 2

Use the following HTML code and save it in .HTML file as **ElementarySchool.html**.

```
<html>
<head>
<title>Untitled Document</title>
</head>
```

```

<body>
<p>Elementary School</p>
<ul>

<li><a href="index.htm">Home</a></li>
<li><a href="schoolinfo/schoolinfo.htm">School Information</a></li>

<li><a href="schoolinfo/calendar.htm">Calendar</a></li>
<li><a href="academics/academics.htm">Academics</a></li>
<li><a href="extracurricular/extracurricularactivities.htm">Extracurricular
Activities</a></li>
<li><a href="students/students.htm">For Students</a></li>
<li><a href="parents/parents.htm">For Parents</a></li>
<li><a href="staff/staff.htm">For Staff</a></li>
<li><a href="staff/schooldevelopment.htm">School Development</a></li>
<li><a href="search/demosearch.htm">Search</a></li>
</ul>
<h1 align="left"> Welcome to our school </h1>
<p align="left">Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy
eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam
voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita
kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem
ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod
tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At
vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd
gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum
dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor
invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero
eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no
sea takimata sanctus est Lorem ipsum dolor sit amet.

</p>
<h1 align="left">School News</h1>
<p align="left">Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper
suscepit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum
iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel
illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio
dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te
feugait nulla facilisi.</p>
<h1>Upcoming Events</h1>
<ul>
<li>Dec 8 ~ Christmas Concert</li>
<li>Dec 19th ~ last day of classes for 2017</li>
<li>Jan 5th ~ first day of classes for 2017</li>

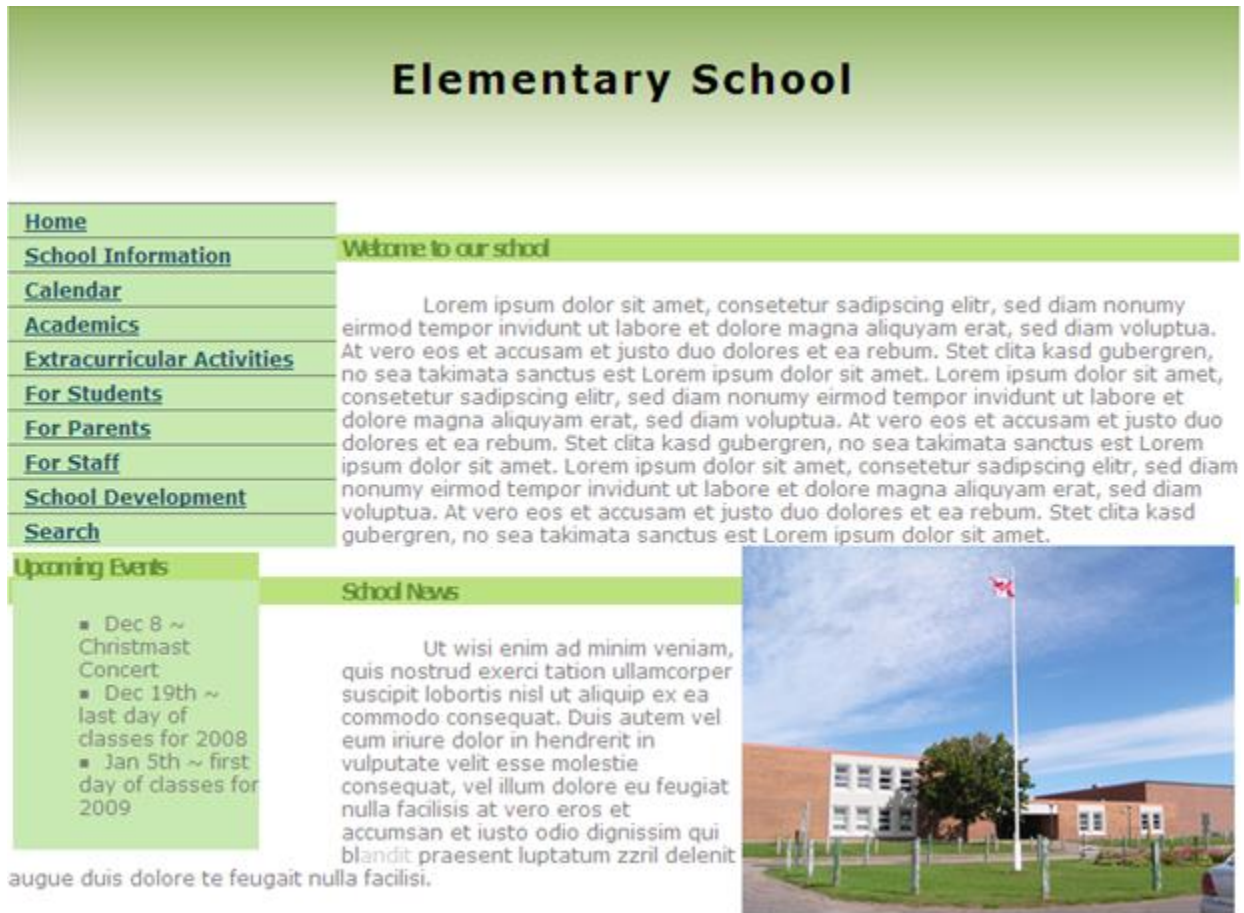
```

```

</ul>
<p align=center>© Name of Elementary | Contact | Site Map | Last Updated:
12/12/2017 </p>
</body>
</html>

```

You are given the file **ElementarySchool.html** which creates the home page for **Elementary School** website. Use the HTML code provided and apply CSS on that HTML. HTML page should look like:



© Name of Elementary | Contact | Site Map | Last Updated: 12/12/2017

Instructions:

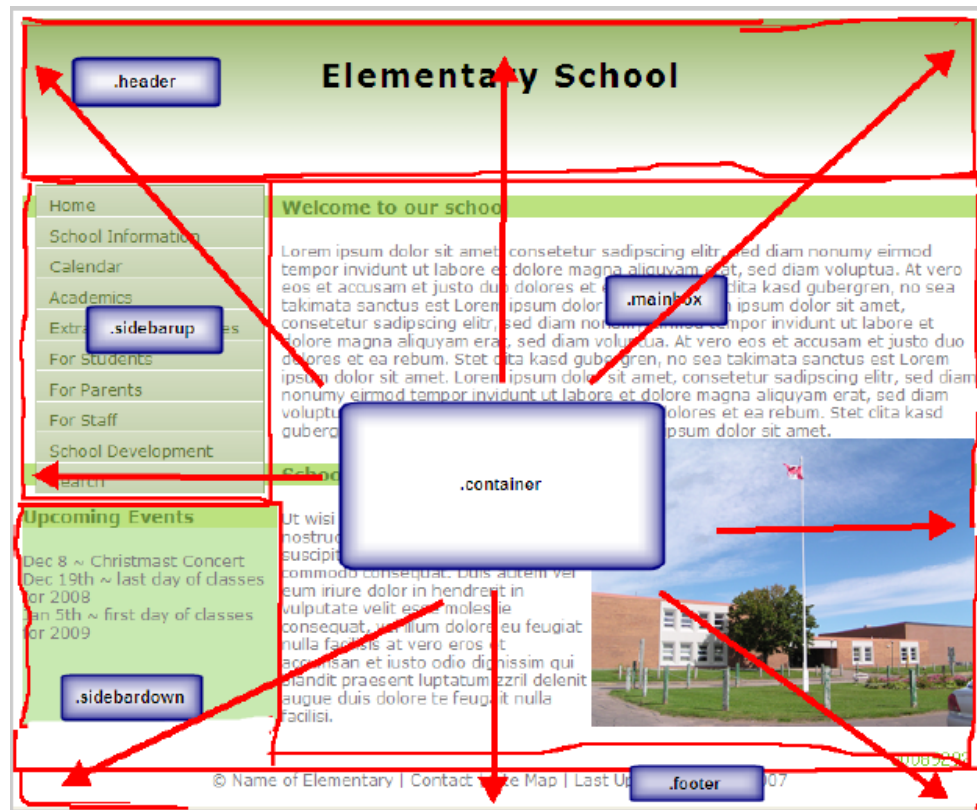
Keep the following organization in your mind while writing CSS. Creating a container to hold everything will accomplish what you did in the HTML section with the main table.

```

/**Layout**/

.container {
    float:none;
    width:750px;
}

```

Header Class:

```
.header {
  font-size: 24px;
  font-weight: bold;
  color: #000000;
  letter-spacing: 2px;
  text-align: center;
  height: 120px;
  width: 750px;
  padding-top: 30px;
  padding-right: 0px;
  padding-bottom: 30px;
  padding-left: 0px;
}
```

The following 2 classes: **sidebarup** and **sidebardown** create the equivalent of two cells which we will ensure make up the left-hand side of the page.

```
.sidebarup {
  text-align: left;
  clear: left;
  float: left;
  width: 200px;
}

.sidebardown {
  width: 200px;
  float: left;
  text-align: left;
  clear: left;
}
```

The **.mainbox** class creates a container which can mimic the main cell of the table created in HTML. The **footer** replaces the bottom cell of the table.